

# TRACK: DATA ANALYTICS

**Completion Rule:** Complete at least 3 tasks from Level 1 or Level 2 combined. To be eligible for a Letter of Recommendation (LOR), complete the maximum number of tasks from both levels.

## LEVEL 1

### TASK 1 · EDA on Retail Sales Data

**Objective:**

Perform a thorough Exploratory Data Analysis on a retail sales dataset to uncover patterns, customer behaviour trends, and actionable business insights.

**Tech Stack:**

Python, pandas, matplotlib, seaborn, Jupyter Notebook

**Feature Checklist:**

- Load dataset and perform initial inspection: shape, column dtypes, null value check
- Descriptive statistics: mean, median, mode, standard deviation for all numerical columns
- Time series analysis: plot monthly and quarterly sales trends using line charts
- Customer demographics analysis: distribution of customer age groups, gender breakdown
- Product analysis: top 10 best-selling products; revenue by product category (bar chart)
- Heatmap: correlation matrix between numerical variables
- At least one additional visualisation of your choice that reveals a non-obvious insight
- Markdown cells throughout the notebook with written observations after each chart
- Conclusion section: at least 3 specific, actionable business recommendations based on findings

**Self-Sourcing Guideline:** Search "retail sales dataset" on Kaggle.com — multiple suitable datasets are available (search terms: "superstore sales", "retail sales data", "e-commerce sales dataset"). Search "pandas EDA tutorial retail sales" and "seaborn time series plot tutorial" on YouTube for technique guidance.

---

### TASK 2 · Customer Segmentation Analysis

**Objective:**

Apply clustering algorithms to segment an e-commerce company's customer base into distinct groups based on purchasing behaviour, enabling targeted marketing strategies.

**Tech Stack:**

Python, pandas, scikit-learn (KMeans), matplotlib, seaborn, Jupyter Notebook

**Feature Checklist:**

- Load dataset and inspect structure; handle missing values and inconsistent data
- Descriptive statistics: average purchase value, purchase frequency, customer lifetime value

- Feature selection: identify 2–3 key behavioural features for clustering (e.g., Recency, Frequency, Monetary — RFM analysis)
- Data normalisation/standardisation before clustering (use StandardScaler)
- Apply K-Means clustering — use the Elbow Method to determine optimal number of clusters (K)
- Visualise clusters using scatter plots (at least 2 feature combinations)
- Profile each cluster: calculate mean feature values per cluster and describe the customer type
- Bar chart: number of customers per cluster
- Insights section: what marketing action would you recommend for each segment?

**Self-Sourcing Guideline:** Search "customer segmentation RFM analysis Python tutorial" on YouTube. For a dataset, search "e-commerce customer data" on Kaggle.com (the "Online Retail" dataset from UCI is a classic choice). Search "KMeans elbow method scikit-learn tutorial" for clustering implementation guidance.

---

## TASK 3 · Cleaning Data

### Objective:

Demonstrate professional-level data cleaning skills by taking a deliberately messy dataset and systematically transforming it into a clean, analysis-ready dataset. Document every decision.

### Tech Stack:

Python, pandas, numpy, Jupyter Notebook

### Feature Checklist:

- Load dataset and produce a "data quality report": count of nulls per column, duplicate rows, data type issues, value range anomalies
- Missing data handling: choose an appropriate strategy for each column (mean/median imputation, mode imputation, forward fill, or row deletion) and justify each choice in a markdown cell
- Duplicate removal: identify and remove duplicate rows; document how many were removed
- Standardisation: normalise inconsistent formatting (e.g., "Male"/"male"/"M" → "Male"; date formats → datetime)
- Outlier detection: use IQR method or Z-score to identify outliers in numeric columns; decide and document whether to cap, remove, or retain each
- Data type correction: ensure all columns have the correct dtype (e.g., dates as datetime, IDs as string, monetary values as float)
- Produce a "before vs. after" summary table: null count, duplicate count, row count, and dtype accuracy — before and after cleaning
- Save the cleaned dataset to a new CSV file

**Self-Sourcing Guideline:** For a messy dataset, search "dirty dataset for data cleaning practice" on Kaggle.com (search tags: "data cleaning"). Good options include the "Titanic dataset", "Housing dataset", or search specifically "messy data cleaning exercise". Search "pandas data cleaning tutorial 2024" on YouTube for comprehensive technique walkthroughs.

---

## TASK 4 · Sentiment Analysis

### Objective:

Build a machine learning model that classifies the sentiment of text data (positive, negative, or neutral), providing insights into public opinion or customer feedback.

### Tech Stack:

Python, pandas, scikit-learn, NLTK or TextBlob, matplotlib/seaborn, Jupyter Notebook

### Feature Checklist:

- Load dataset and inspect class distribution (positive/negative/neutral counts)
- Text preprocessing pipeline: lowercase, punctuation removal, stopword removal, tokenisation, optional stemming/lemmatization
- Feature extraction: TF-IDF Vectorizer (explain its purpose in a markdown cell)
- Train/test split (80/20)
- Train at least 2 classifiers: Naive Bayes + one of (SVM, Logistic Regression, or a simple LSTM if comfortable with deep learning)
- Evaluation: accuracy, precision, recall, F1-score, confusion matrix for each model
- Visualisation: bar chart of sentiment distribution in the dataset; WordCloud for each sentiment class
- Error analysis: show 5 examples the model misclassified and discuss why
- Conclusion: which model performed best and what real-world application could this serve?

**Self-Sourcing Guideline:** Search "sentiment analysis dataset" on Kaggle.com (good options: "Twitter Sentiment Analysis", "Amazon Product Reviews", "IMDB Movie Reviews"). Search "Python sentiment analysis NLP tutorial scikit-learn" on YouTube. Reference the NLTK documentation (nltk.org) for preprocessing and TextBlob's documentation for a simpler rule-based baseline.

## LEVEL 2

## TASK 1 · Predicting House Prices with Linear Regression

### Objective:

Build and evaluate a linear regression model that predicts house prices based on features such as area, location, number of rooms, and age. Develop end-to-end skills from data cleaning through to model interpretation.

### Tech Stack:

Python, pandas, scikit-learn, matplotlib, seaborn, Jupyter Notebook

### Feature Checklist:

- Load dataset and perform EDA: null check, descriptive statistics, distribution of the target variable (price)
- Feature selection discussion: identify which features are likely predictors; explain reasoning in a markdown cell

- Handle missing values and encode categorical features (One-Hot Encoding)
- Correlation heatmap: identify features most correlated with house price
- Train/test split (80/20)
- Train a Linear Regression model using scikit-learn
- Evaluate model using: Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and  $R^2$  score
- Scatter plot: actual prices vs. predicted prices (the closer to a diagonal line, the better)
- Residual plot: check that residuals are randomly distributed (no systematic pattern)
- Coefficient analysis: which features have the highest positive/negative impact on price?
- (Bonus) Compare Linear Regression against a Ridge or Lasso regularised model

**Self-Sourcing Guideline:** Search "house price prediction dataset" on Kaggle.com (the "Ames Housing Dataset" and "House Prices: Advanced Regression Techniques" competition datasets are both excellent). Search "linear regression house price prediction Python tutorial" on YouTube. Reference the scikit-learn documentation for `LinearRegression`, `mean_squared_error`, and `r2_score`.

## TASK 2 · Wine Quality Prediction

### Objective:

Train and compare multiple classification models to predict the quality score of wine (typically a scale of 3–8) based on its physicochemical properties such as acidity, density, and alcohol content.

### Tech Stack:

Python, pandas, numpy, scikit-learn (Random Forest, SGD, SVC), seaborn, matplotlib, Jupyter Notebook

### Feature Checklist:

- Load the Wine Quality dataset and inspect structure; check class distribution of quality scores
- EDA: distribution plots for all chemical features; correlation heatmap
- Discuss class imbalance: are certain quality scores underrepresented? How does this affect modelling?
- Feature engineering: consider binning quality scores into binary (good/bad) or 3-class (low/medium/high) groups and justify the decision
- Train/test split with stratification to preserve class ratios
- Train 3 classifiers: Random Forest, Stochastic Gradient Descent (SGD), and Support Vector Classifier (SVC)
- Evaluate each: accuracy, classification report, confusion matrix
- Feature importance chart for the Random Forest model
- Comparison table: summarise the performance of all 3 models side-by-side
- Conclusion: which model is most suitable for deployment and why?

**Self-Sourcing Guideline:** Search "Wine Quality dataset UCI" — it is available on both Kaggle.com and the UCI Machine Learning Repository ([archive.ics.uci.edu/dataset/186/wine+quality](https://archive.ics.uci.edu/dataset/186/wine+quality)) for free. Search "wine quality prediction Python Random Forest SVM tutorial" on YouTube. Reference scikit-learn docs for `RandomForestClassifier`, `SGDClassifier`, and `SVC`.

---

## TASK 3 · Fraud Detection

### Objective:

Build a machine learning pipeline to detect fraudulent financial transactions from a heavily imbalanced dataset, addressing class imbalance as a core challenge.

### Tech Stack:

Python, pandas, scikit-learn, imbalanced-learn (SMOTE), matplotlib, seaborn, Jupyter Notebook

### Feature Checklist:

- Load dataset and analyse class imbalance: what percentage of transactions are fraudulent?
- EDA: distribution of transaction amounts for fraud vs. non-fraud; time-of-day analysis
- Discuss why standard accuracy is a misleading metric for imbalanced fraud datasets (explain in a markdown cell)
- Apply a class imbalance handling technique: SMOTE oversampling, undersampling, or `class_weight='balanced'`
- Train/test split (ensure fraud cases appear in both splits using stratification)
- Train at least 2 models: Logistic Regression + one of (Decision Tree, Random Forest, or XGBoost)
- Evaluate using: Precision, Recall, F1-Score, and AUC-ROC curve (not just accuracy)
- Explain which metric matters most for fraud detection and why (Recall vs. Precision trade-off)
- Feature importance or coefficient analysis
- Discuss scalability: how would this model handle 1 million transactions per hour?

**Self-Sourcing Guideline:** Search "credit card fraud detection dataset" on Kaggle.com — the "Credit Card Fraud Detection" dataset (284,807 transactions, 492 fraudulent) is a benchmark dataset specifically designed for this task. Search "fraud detection Python imbalanced dataset SMOTE tutorial" on YouTube. Reference the imbalanced-learn documentation (imbalanced-learn.org) for SMOTE implementation.

---

## TASK 4 · Unveiling the Android App Market (Google Play Store Analysis)

### Objective:

Perform a comprehensive data analysis of the Google Play Store ecosystem — cleaning messy real-world data, exploring app categories, analysing ratings and pricing trends, and conducting sentiment analysis on user reviews.

### Tech Stack:

Python, pandas, numpy, matplotlib, seaborn, TextBlob or VADER, Jupyter Notebook

### Feature Checklist:

- Load the Play Store apps dataset and the user reviews dataset separately
- Data cleaning: fix incorrect data types (e.g., "Installs" stored as "10,000+" strings), handle nulls, remove duplicates
- Category analysis: bar chart of app distribution across categories; which categories are most saturated?

- Ratings analysis: distribution of app ratings; average rating by category
- Size and installs analysis: scatter plot of app size vs. number of installs; is there a correlation?
- Pricing analysis: paid vs. free app distribution; price distribution for paid apps; revenue estimate by category
- Sentiment analysis on user reviews: classify reviews as positive/negative/neutral using TextBlob or VADER
- Sentiment by category: which app categories have the most positive/negative user sentiment?
- At least one interactive visualisation using plotly (optional but strongly recommended)
- Conclusion: 3 data-driven insights for a developer planning to launch a new app

**Self-Sourcing Guideline:** Search "Google Play Store dataset" on Kaggle.com — the dataset "Google Play Store Apps" is publicly available. Search "Google Play Store EDA Python tutorial" on YouTube. For sentiment analysis, search "VADER sentiment analysis Python tutorial" — VADER is pre-built for social media text and requires no training.

## TASK 5 · Autocomplete and Autocorrect Data Analytics

### Objective:

Analyse the efficiency and accuracy of autocomplete and autocorrect algorithms using NLP techniques. Implement and compare multiple approaches for text prediction and spelling correction on a real text dataset.

### Tech Stack:

Python, pandas, NLTK, pyspellchecker or textdistance, collections, matplotlib, Jupyter Notebook

### Feature Checklist:

- Collect or download a large text corpus (see Self-Sourcing Guideline)
- NLP preprocessing: tokenisation, lowercasing, punctuation removal, stopword removal
- Autocomplete implementation: build a frequency-based n-gram model (bigram or trigram) that predicts the next word given a prefix
- Test autocomplete on at least 10 input prefixes and display top 3 predictions per prefix
- Autocorrect implementation: implement edit-distance based correction using pyspellchecker or custom Levenshtein distance logic
- Test autocorrect on at least 20 deliberately misspelled words and measure correction accuracy
- Performance metrics: define and calculate precision and recall for both autocomplete and autocorrect
- Algorithm comparison: compare at least 2 approaches (e.g., frequency-based vs. neural n-gram, or two different edit-distance algorithms)
- Visualisation: bar chart of top 20 most frequent words; confusion matrix for autocorrect (correct/incorrect before and after)
- Discussion: what are the limitations of your implementation vs. production systems like Google Keyboard?

**Self-Sourcing Guideline:** For a text corpus, download "Project Gutenberg books" (gutenberg.org — free public domain texts) or search "large text corpus NLP dataset" on Kaggle.com (the "Enron Email Dataset" or "Wikipedia Text Corpus" work well). Search "Python autocomplete n-gram model tutorial" and "Python spell checker

pyspellchecker tutorial" on YouTube. Reference the NLTK documentation for ngrams and FreqDist.

---